Generalizability of GNNs for TSP

Reid Sewell (rs273) Raha Gharehdaghi (rg152)

Abstract

While Graph Neural Networks (GNNs) offer a fast, data-driven alternative to classical solvers for the Traveling Salesperson Problem (TSP), they consistently struggle to generalize to larger instance sizes and unseen spatial structures. This project investigates the extent to which training on structurally diverse datasets with variable node counts improves a model's zero-shot extrapolation. An encoder-decoder GNN was trained on synthetic datasets comprising random, regular, and cluster graph distributions with sizes ranging from 25 to 75 nodes. To maintain computational tractability and preserve optimal routing edges, kNN-based sparsification was implemented. The solver's generalization was then evaluated on larger mixed synthetic graphs and real-world TSPLib instances containing up to 1000 nodes.

1 Introduction

The Traveling Salesperson Problem (TSP) is one of the most widely studied problems in combinatorial optimization. Given a list of cities and the distances between them, what is the shortest possible route that visits each city exactly once and returns to the origin? Despite this simple premise, TSP is NP-hard. As the number of cities increases, the possible routes grow factorially. This makes it computationally infeasible to find guaranteed optimal solutions for large-scale, real-world applications—such as logistics, supply chain management, and vehicle routing, using exact algorithms. To bypass this computational bottleneck, the field has historically relied on handcrafted approximation algorithms.

Recently, however, data-driven approaches using deep learning have emerged as a powerful alternative. Specifically, Graph Neural Networks (GNNs) have shown immense potential. Because TSP is inherently a graph-based problem, GNNs are naturally suited to tackle it. Using an encoder-decoder architecture, the network processes the spatial relationships within the graph and outputs the probability of each edge belonging to the optimal tour. This approach can drastically reduce the time it takes to generate a high-quality solution during inference.

Despite these advantages, deploying GNN-based solvers in practical scenarios is heavily bottlenecked by their inability to generalize. Most neural TSP solvers are trained on highly controlled, synthetic datasets consisting of small, uniformly distributed graphs with a fixed number of nodes. Consequently, the models overfit to the specific spatial density and scale of their training data. When these models are evaluated on much larger instances, or on datasets with irregular, clustered spatial distributions akin to real-world data, their routing logic breaks down. Bridging this gap—ensuring a model can generalize across both different instance sizes and unseen structural distributions—remains a critical challenge in neural combinatorial optimization.

This project investigates the extent to which introducing structural diversity and variable scale during the training phase can improve the zero-shot generalization of a GNN-based TSP solver. Rather than training on a single, static graph type, this methodology trains models on a mixed dataset comprising random, regular, and cluster graph distributions. To maintain computational tractability during training, kNN-based sparsification. The remainder of this report is organized as follows: Section 2 provides background on the use of GNNs in combinatorial optimization and details current generalization challenges. Section 3 outlines the methodology, including data generation,

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

sparsification, and the training pipeline. Section 4 presents the analysis and discussion of the solver's performance across different scales and structures. Finally, Section 5 concludes the report.

2 Background

The Traveling Salesperson Problem (TSP) is a fundamental benchmark for combinatorial optimization. While classical solvers are highly effective, the computational cost of large scale routing drives the search for faster alternatives. Graph Neural Networks (GNNs) have emerged as a promising solution. However, deploying these neural solvers in practice requires overcoming critical challenges in how they scale to larger problems and adapt to diverse, real world data.

2.1 GNN usage in solving Combinatorial Optimization Problems

Graph Neural Networks (GNNs) have become a popular approach for tackling NP-hard combinatorial optimization problems [1]. Rather than relying on computationally expensive exact algorithms, modern neural solvers learn to identify optimal structural patterns directly from the problem data. This mechanism is typically framed as an encoder-decoder task: a GNN acts as an encoder by processing the underlying graph structure (nodes and edges) into continuous embeddings, and an autoregressive or non-autoregressive decoder utilizes these embeddings to construct a solution, such as selecting the optimal set of variables [6]. Because the Traveling Salesperson Problem (TSP) is a canonical NP-hard CO problem that is natively represented as a graph, this exact neural mechanism is directly applicable to it. The broader CO encoder-decoder pipeline seamlessly transitions to routing, where the decoder simply predicts the probability of an edge belonging to the optimal tour [5]. By leveraging these general CO advancements, GNNs can generate high-quality TSP solutions in a fraction of the time required by traditional methods during inference.

Recent work in neural combinatorial optimization has seen a significant shift toward masking paradigms like MaskCO [2] and LMask [7] are the two recent papers from ICLR 2026, which reframe problem-solving as a self-supervised task. MaskCO introduces a "masked generation" approach that treats optimal solutions as composite structures, strategically masking portions of them during training to force the model to internalize fine-grained structural dependencies. This paradigm has demonstrated remarkable results, reducing the optimality gap by more than 99% and achieving a 10x speedup on problems like the Traveling Salesman Problem (TSP) compared to previous state-of-the-art neural solvers. Similarly, LMask addresses complex feasibility constraints in routing problems through a "LazyMask" mechanism, which uses backtracking and dynamic masking to ensure solution validity. These approaches signal a promising future direction by moving away from monolithic instance-level construction toward more scalable, localized representation learning that can progressively refine solutions through iterative mask-and-reconstruct loops.

2.2 Graph Structures found in TSP

Training neural CO solvers requires massive amounts of data, making synthetic dataset generation a necessity. The majority of literature generates these instances by uniformly sampling points in a 2D Euclidean space (random graphs). However, standard random graphs fail to reflect the structural diversity and complexity of real-world routing problems. Real-world datasets, such as those compiled in the standard TSPLib benchmark [9, 3], often contain highly irregular spatial distributions that standard random generators cannot replicate.

To better capture these varying spatial properties and study their effects on model performance, Stodola and Ščurek identify three synthetic graph distributions commonly used in literature: random graphs, regular graphs, and cluster graphs [10]. These synthetic graphs are defined by the specific methods used to place points within a 2D rectangular space. In random graphs, vertices are scattered uniformly across the space. In regular graphs, vertices are arranged in structured, grid-like patterns. In cluster graphs, vertices are grouped together into dense, isolated pockets. By utilizing the algorithmic generation methods provided by the authors for each of these graph types, researchers can construct structurally diverse datasets to systematically test a neural solver's adaptability.

2.3 Generalizability of GNNs

Despite performing well on small, fixed-distribution datasets, GNN-based TSP solvers struggle with generalization across two main axes: scale and structure.

First, models trained on fixed-size graphs (e.g., exactly 50 nodes) severely overfit to that specific spatial density. When evaluated on larger instances, performance drops rapidly due to the increased graph diameter [5]. Furthermore, scaling up introduces computational limits. A fully connected TSP graph has $O(N^2)$ edges, making large instances intractable. This necessitates graph sparsification to reduce edges to a more reasonable level, though heuristic sparsification risks accidentally removing edges that comprise the optimal tour [5].

To counter the scaling bottleneck, some researchers propose divide-and-conquer techniques that decompose arbitrarily large graphs into smaller, manageable subgraphs [4]. While this allows models to process thousands of nodes, it fails to address the second major axis of generalization: structural distribution shifts. Subgraphs inherently retain many of the spatial features of their parent graphs. Consequently, if a model is trained exclusively on uniform, random 2D Euclidean subgraphs, it will still fail to transfer to subgraphs extracted from irregular or highly clustered real-world datasets [3].

3 Methodology

This paper explores how the synthetic training data affects the generalizability of a GNN-based TSP solver. We use the Neural combinatorial optimization pipeline presented by Joshi et al. as the basis for our work. This pipeline includes three phases: the problem definition, graph embedding, and solution decoding.

The problem definition is the method by which synthetic data is generated. All datasets are made from 10000 graphs. The datasets include a random graph dataset, a regular graph dataset, a cluster graph dataset, and a mixed dataset which includes all 3 graph types. Stodola and Ščurek [10] proposed the mixed dataset to improve genralizability, as they believed a neural learner which had seen many different graph types during training would be able to better adapt during inference. Each of these datasets is generated first with only 50 nodes, and a corresponding dataset is generated with between 25-75 nodes. As discussed by Joshi et al., static numbers of nodes are typically used for dataset creation due to ease, but using a range of nodes could lead to more generalizable solvers [5]. Joshi et al. also suggest kNN-based graph sparsification to reduce training time and intractability [5]. We chose to sparsify the graphs by keeping 20% of nearest edges to each node.

A GNN-based TSP solver will be trained on each of these datasets, and their performance will be compared. The GNN-based solver comprises the second two phases of the pipeline: graph embedding and solution decoding. Graph embedding is done with a GNN, which is implemented using GINEConv layers in pyTorch. Solution decoding is done with an autoregressive decoder. Forced teacher learning is used to ensure stability during training, since without it the model's errors would compound early on.

The ground truth solutions for each TSP were found with or-tools [8]. The optimal route was computed prior to performing sparsification, and so any removed optimal edges were added back to the graph afterwards to ensure the correct solution was not accidentally removed. The main metric used is the percent away from optimal, which measures how far away the length of the model's predicted graph is when compared to the optimal graph. All code for this project was done in python, and implemented in a jupyter notebook which can be found at this github repository.

4 Analysis and Discussion

The loss during training can be seen in Figure 1 and the validation percentage from optimal can be seen in Figure 2.

The GNN-based TSP solver is evaluated on its ability to solve two test datasets. The first dataset is TSPLib, a common benchmark dataset for TSP solvers [9]. All models were evaluated on TSPLib instances up to 100 nodes, 250 nodes, and 1000 nodes. Results on TSPLib can be seen in Figure 3, and the median percentage away from optimal for each model on each dataset size can be seen in

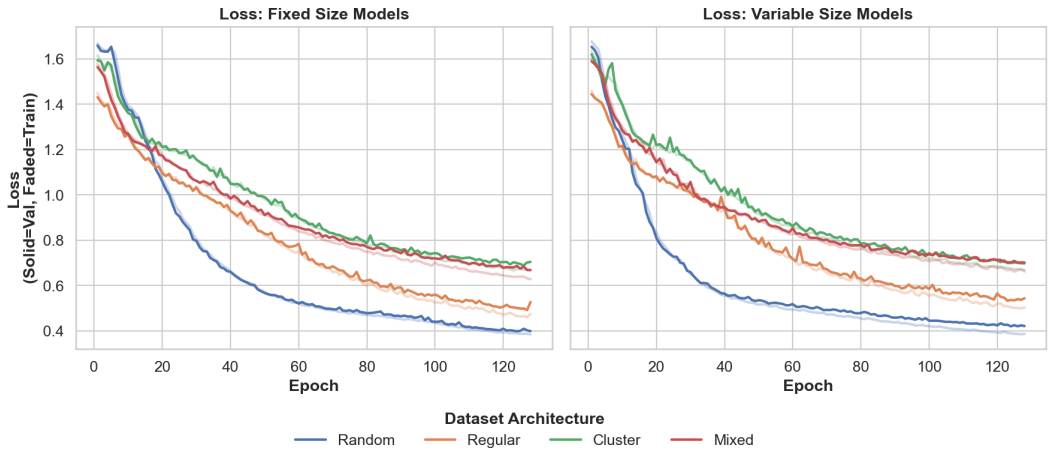

Figure 1: Loss during training, separated by fixed and variable size models for readability. Training loss is represented by the faded lines, and validation loss is represented by the solid lines.

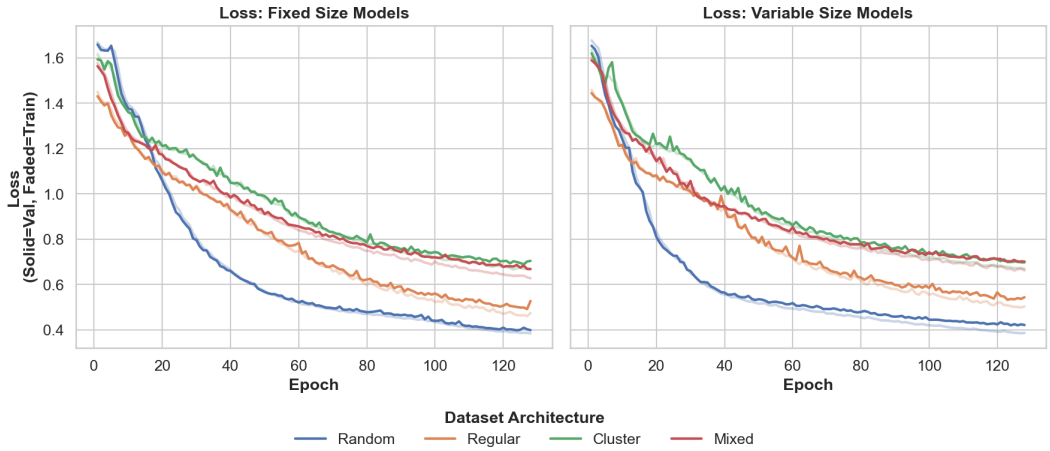

Figure 2: Validation percent from optimal during training, separated by fixed and variable size models for readability.

Table 1. The second dataset is mixed synthetic graphs with 50, 100, 250, and 500 nodes[1]. Results on the mixed synthetic data can be seen in Figure 4, and the median percentage away from optimal for each model on each dataset size can be seen in Table 2.

Table 1: Median percent (%) from optimal per model per size

model \ max_nodes	**100**	**250**	**1000**
cluster_n_weights.pt	341.68	409.38	539.63
cluster_var_weights.pt	389.74	473.64	599.68
mixed_n_weights.pt	363.46	446.04	535.05
mixed_var_weights.pt	355.27	463.72	535.12
random_n_weights.pt	343.11	488.67	621.97
random_var_weights.pt	**282.71**	**360.39**	**399.06**
regular_n_weights.pt	350.53	490.66	656.30
regular_var_weights.pt	359.13	458.96	552.04

[1]We did try up to 1000 nodes, intending to match the breakpoints for the evaluations on TSPLib, but this caused integer overflow issues in or-tools graph solver.

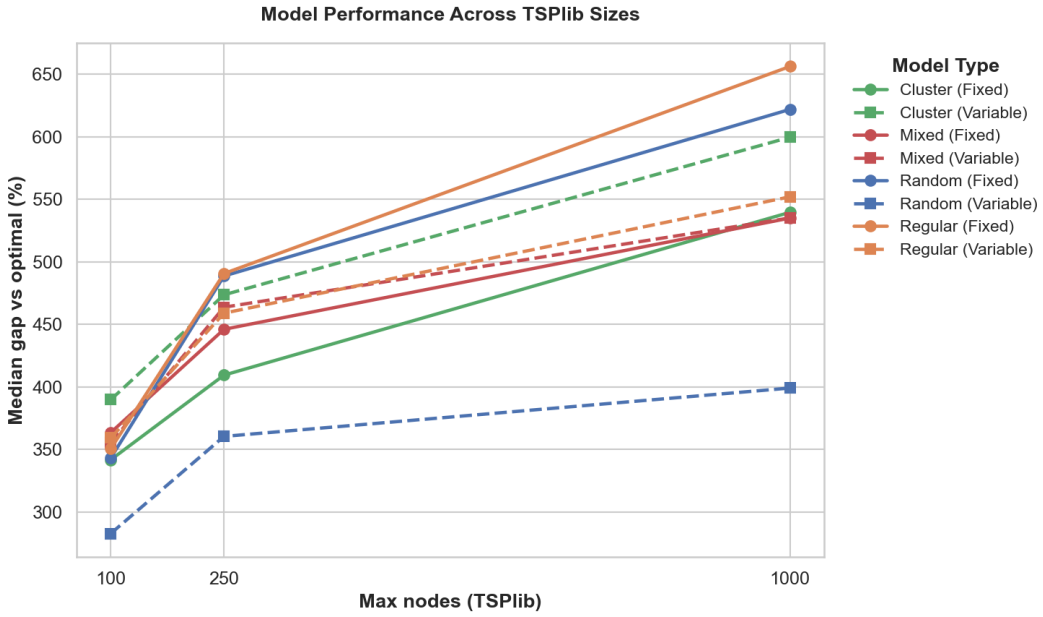

Figure 3: Median percent from optimal for each model on TSPLib graphs of up to varying sizes.

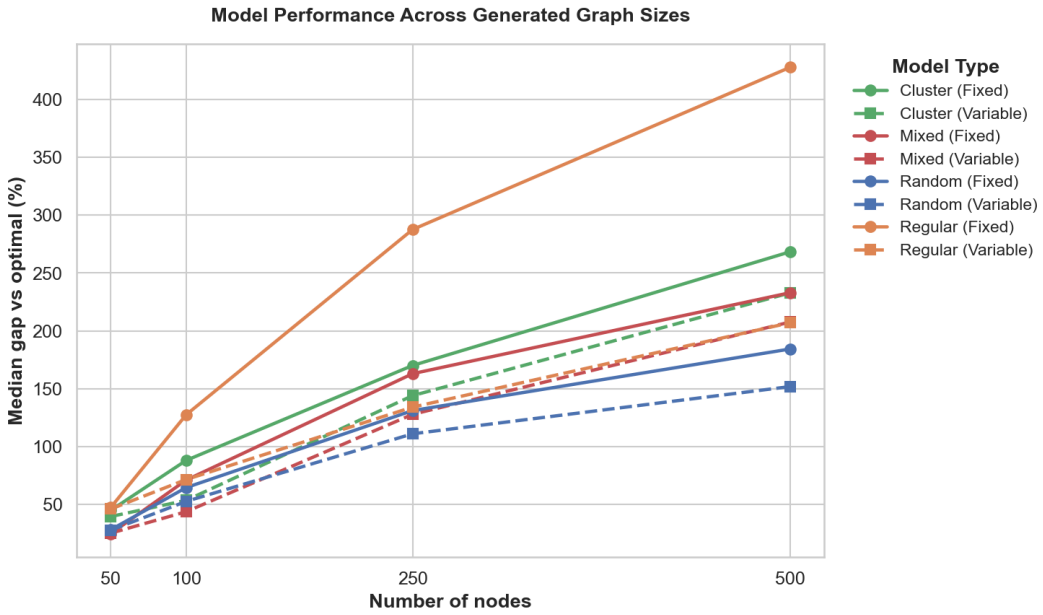

Figure 4: Median percent from optimal for each model on mixed synthetic graphs of varying sizes.

Table 2: Median percent (%) from optimal per model per size

model \ n_nodes	**50**	**100**	**250**	**500**
cluster_n_weights.pt	44.85	88.09	170.10	268.30
cluster_var_weights.pt	39.53	53.55	144.02	232.79
mixed_n_weights.pt	**24.74**	71.09	162.99	232.96
mixed_var_weights.pt	25.13	**43.72**	127.85	207.62
random_n_weights.pt	28.13	64.58	131.10	184.31
random_var_weights.pt	27.72	52.64	**111.04**	**151.81**
regular_n_weights.pt	47.43	127.55	287.60	427.73
regular_var_weights.pt	46.00	71.54	134.09	206.96

Across both datasets, the models performed very similarly on graphs close to the size they were trained on. As size increased in both datasets, the model trained on random graphs of varying sized had the best median percentage from optimal. For TSPLib, the next best performers were clustered and mixed models trained on fixed size graphs, which both outperformed the remaining variable graphs. On the mixed synthetic dataset, each variable graph outperformed its fixed-size counterpart, though the random fixed-size model outperformed some variable models.

4.1 Analysis of Improvements

On real world graphs such as TSPLib, the model trained on random and variable size graphs performed best, but no clear trend of any kind can be discerned as graph size increases. This points to a limitation of the models more than of the methods, and it is likely the architecture used did not have enough parameters to learn well, despite the different training methods. The best conclusion to be drawn here is that for models being trained with limited compute, a dataset of random and variable sized graphs provides the generalization to larger graph sizes.

In comparison, the synthetic graphs do paint a clear trend. Each variable-sized model far outperformed its fixed-size counterpart, even as graph sizes increased. Thus, we can say that training on graphs of variable size does improve generalizability for this test environment. But, some variable-sized models were also outperformed by the fixed-sized random graph model.

It is hard to say why the models trained on random graphs outperformed the remaining models as graph size increased. It is possible that, due to the unstructured nature of the random graph generation, it was able to be less reliant on structures that are harder to follow in larger graph instances.

As for the models trained on mixed graph structures, they struggled more than expected, likely due to the same compute limitations addressed before. It is likely that the mixed model needed more parameters than other models in order to recognize and adapt to the different graph structures it was shown.

5 Conclusion

The Traveling Salesperson Problem (TSP) is a fundamental benchmark in combinatorial optimization where finding the shortest route between cities is NP-hard. While Graph Neural Networks (GNNs) offer a fast alternative to classical solvers by learning spatial patterns, they typically struggle to generalize. Most neural solvers are trained on small, fixed-size synthetic datasets, causing them to overfit and fail when applied to the irregular distributions and larger scales found in real-world benchmarks like TSPLib.

This project investigated the extent to which introducing structural diversity and variable scale during training improves the zero-shot generalization of GNN-based TSP solvers. The results demonstrate a clear trend on synthetic graphs: models trained on variable-sized datasets consistently and significantly outperformed their fixed-size counterparts, even as graph sizes increased. By moving away from highly controlled, fixed-distribution datasets , this approach successfully mitigated the tendency of neural solvers to overfit to a specific spatial density. Ultimately, incorporating variable node counts during training is a highly effective methodology for improving generalization across different scales.

Interestingly, our evaluation on TSPLib showed that the model trained on random, variable-sized graphs performed the best on real-world data. While standard random graphs lack the realistic clustering of actual cities, the uniform unpredictability of placing nodes anywhere may actually be a stronger testing environment than we expected. Rather than allowing the model to overfit to specific cluster patterns or mixed distributions, training on random noise with variable sizes might force the model to learn more robust, generalized routing logic. While architectural limitations likely bottlenecked performance on the largest graphs, this suggests that random, variable-scale training data provides a surprisingly effective baseline for zero-shot generalization.

6 Contributions

Both project members contributed to all portions of this project. The majority of background research and discussion was done by Raha Gharehdaghi. The majority of code was done by Reid Sewell. Both project members contributed equally to analyzing and reflecting on results.

References

[1] Furkan Cantürk, Taha Varol, Reyhan Aydoğan, and Okan Örsan Özener. Scalable primal heuristics using graph neural networks for combinatorial optimization. *Journal of Artificial Intelligence Research*, 80:327–376, 2024.

[2] Lvda Chen, Yang Li, and Junchi Yan. Maskco: Masked generation drives effective representation learning and exploiting for combinatorial optimization. In *The Fourteenth International Conference on Learning Representations*, 2026.

[3] Shengyu Feng, Weiwei Sun, Shanda Li, Ameet Talwalkar, and Yiming Yang. A comprehensive evaluation of contemporary ml-based solvers for combinatorial optimization. *arXiv preprint arXiv:2505.16952*, 2025.

[4] Zhang-Hua Fu, Kai-Bin Qiu, and Hongyuan Zha. Generalize a small pre-trained model to arbitrarily large tsp instances. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 7474–7482, 2021.

[5] Chaitanya K Joshi, Quentin Cappart, Louis-Martin Rousseau, and Thomas Laurent. Learning the travelling salesperson problem requires rethinking generalization. *Constraints*, 27(1):70–98, 2022.

[6] Wouter Kool, Herke Van Hoof, and Max Welling. Attention, learn to solve routing problems! *arXiv preprint arXiv:1803.08475*, 2018.

[7] Tianyou Li, Haijun Zou, Jiayuan Wu, and Zaiwen Wen. Lmask: Learn to solve constrained routing problems with lazy masking. *arXiv preprint arXiv:2505.17938*, 2025.

[8] Laurent Perron and Vincent Furnon. OR-Tools. `https://developers.google.com/optimization/`, 2024.

[9] Gerhard Reinelt. Tsplib–a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991.

[10] Petr Stodola and Radomír Ščurek. Using machine learning in combinatorial optimization: Extraction of graph features for travelling salesman problem. *Knowledge-Based Systems*, 314:113216, 2025.